

3.1 Công nghệ sử dụng

Chương này trình bày chi tiết các công nghệ, nền tảng và cơ sở lý thuyết được lựa chọn để hiện thực hóa hệ thống chia sẻ thực phẩm đã được đặc tả ở Chương 2.

3.2 Tổng quan kiến trúc và ánh xạ yêu cầu

Hệ thống bao gồm ba thành phần chính: ứng dụng di động dành cho ba vai trò người dùng (Người cho (Donor), Người nhận (Receiver), Tình nguyện viên (Volunteer)), website dành cho quản trị viên (admin) và máy chủ phụ trợ (backend) cung cấp REST API kết hợp kênh truyền thông thời gian thực. Bảng ?? dưới đây thể hiện sự ánh xạ từ các yêu cầu cốt lõi ở Chương 2 sang nhóm công nghệ được sử dụng để giải quyết.

Yêu cầu	Nhóm công nghệ được chọn
Ứng dụng di động đa nền tảng (Android) dùng chung một mã nguồn	React Native + Expo, TypeScript
Lưu trữ dữ liệu bài đăng, đơn giao, hồ sơ linh hoạt	MongoDB + Mongoose
Giao tiếp client-server theo nghiệp vụ	Express (REST API), Axios
Trò chuyện và cập nhật trạng thái đơn thời gian thực	Socket.IO
Đăng nhập an toàn, phân quyền theo vai trò	JWT, bcrypt, Middleware
Chọn điểm lấy/giao trên bản đồ, tính khoảng cách	react-native-maps, expo-location
Tải ảnh bài đăng, ảnh chat, ảnh đại diện	Multer, expo-image-picker
Tự động hết hạn hoặc xác nhận đơn theo thời gian	Tác vụ định kỳ (cron) phía server
Quản lý trạng thái ứng dụng, đa ngôn ngữ	Zustand, mô-đun i18n tự xây dựng
Triển khai và vận hành	Docker

Bảng 3.1: Ánh xạ yêu cầu hệ thống và nhóm công nghệ

3.3 Cơ sở lý thuyết nền tảng

Trước khi đi sâu vào các công cụ cụ thể, mục này tóm tắt các lý thuyết nền tảng mà hệ thống dựa vào. Đây là khối kiến thức cốt lõi được vận dụng

để giải quyết các bài toán đã đặt ra ở Chương 2.

3.3.1 Kiến trúc REST và mô hình client–server

REST (Representational State Transfer) là phong cách kiến trúc do R. Fielding đề xuất, trong đó tài nguyên được định danh bằng URL, thao tác qua các phương thức HTTP (GET, POST, PUT, DELETE) và mỗi yêu cầu là *không trạng thái*, tức là máy chủ không lưu trữ phiên giữa các lần gọi.

Hệ thống áp dụng REST cho toàn bộ nghiệp vụ: mỗi yêu cầu đều tự mang đủ thông tin xác thực, cho phép máy chủ dễ dàng mở rộng theo chiều ngang mà không cần chia sẻ phiên. Đây là cơ sở lý thuyết đáp ứng trọn vẹn yêu cầu "giao tiếp client–server theo nghiệp vụ".

3.3.2 Giao tiếp thời gian thực: WebSocket và mô hình hướng sự kiện

Trong các ứng dụng có yêu cầu cập nhật dữ liệu tức thời, cơ chế HTTP truyền thống theo mô hình yêu cầu–phản hồi tỏ ra hạn chế do không hỗ trợ giao tiếp hai chiều liên tục. Do đó, hệ thống sử dụng WebSocket để thiết lập kết nối hai chiều bền vững giữa client và server, cho phép máy chủ chủ động gửi dữ liệu khi có sự kiện phát sinh.

Trên nền tảng WebSocket, hệ thống áp dụng mô hình hướng sự kiện kết hợp cơ chế publish/subscribe theo nhóm (room). Cụ thể: (i) Mỗi người dùng được gán vào một phòng riêng theo định danh `user:<id>`, trong khi mỗi cuộc hội thoại tương ứng với một phòng `conversation:<id>`. Điều này giúp giới hạn phạm vi truyền tin, chỉ gửi dữ liệu đến các client liên quan. (ii) Các thao tác trong hệ thống được mô hình hóa dưới dạng sự kiện như `chat:send`, `chat:typing`, `chat:read`, giúp tách biệt rõ giữa các loại hành vi và tăng tính mở rộng. (iii) Kết nối WebSocket được xác thực bằng JWT ngay trong quá trình thiết lập kết nối (handshake), nhằm đảm bảo chỉ các người dùng hợp lệ mới có thể tham gia trao đổi dữ liệu.

Lý thuyết này giải quyết triệt để yêu cầu trò chuyện và cập nhật trạng thái đơn hàng theo thời gian thực. Phương án giả lập realtime trên HTTP đã bị loại bỏ do có độ trễ cao và gây lãng phí tài nguyên máy chủ.

3.3.3 Định vị địa lý và tính khoảng cách

Vị trí người dùng được biểu diễn bằng cặp tọa độ **vĩ độ/kinh độ** theo hệ quy chiếu **WGS-84**, chuẩn tọa độ phổ biến của GPS và các dịch vụ bản đồ. Để giải quyết bài toán lọc bài đăng/đơn hàng theo bán kính và ước lượng quãng đường, hệ thống áp dụng hai phương pháp: (i) **Khoảng cách đường**

chim bay (Công thức Haversine) MovableTypeLatLong: Dùng để tính khoảng cách giữa hai điểm trên mặt cầu Trái Đất (với bán kính ≈ 6371 km). Công thức được áp dụng như sau:

$$\begin{aligned}a &= \sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}, \\d &= 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})\end{aligned}\tag{3.1}$$

trong đó φ là vĩ độ, λ là kinh độ (đã chuyển sang radian). Đây là phép toán nội bộ, tốc độ cao, không phụ thuộc mạng, dùng để sắp xếp danh sách đơn theo khoảng cách, và lọc đối tượng nhận thông báo đầy: 10 km cho donor, 20 km cho tình nguyện viên."

(ii) **Khoảng cách đường bộ thực tế (Google Distance Matrix API)**
GoogleDistanceMatrixAPI: Hệ thống gọi API để lấy quãng đường di chuyển thực tế. Dữ liệu được kết hợp bộ nhớ đệm (cache) với thời gian sống (TTL) 10 phút và làm tròn tọa độ ~ 111 m để tối ưu chi phí hạn ngạch. Nếu gặp sự cố mạng, hệ thống tự động dự phòng bằng công thức Haversine.

Lưu ý: Các tọa độ lỗi (0, 0) — rơi vào vị trí Vịnh Guinea — được hệ thống tự động loại bỏ để tránh sai số hiển thị.

3.3.4 Lý thuyết bảo mật: JWT và hàm băm mật khẩu

(i) **JSON Web Token:** Một token bao gồm ba phần (header, payload, signature). Phần chữ ký được tạo bằng thuật toán HMAC, kết hợp hàm băm với khóa bí mật nhằm bảo toàn tính toàn vẹn [22]. Máy chủ chỉ cần xác minh chữ ký bằng JWT_SECRET là có thể cấp quyền, hoàn toàn phù hợp với mô hình không trạng thái. (ii) **Hàm băm mật khẩu một chiều:** Mật khẩu luôn được băm trước khi lưu trữ. Thuật toán bcrypt bổ sung chuỗi ngẫu nhiên (chống tấn công rainbow table) và hệ số công (work factor) nhằm làm chậm quá trình băm, khiến việc dò tìm mật khẩu trở nên bất khả thi trong thời gian ngắn.

3.4 Lựa chọn công nghệ di động và máy chủ

3.4.1 Nền tảng ứng dụng di động

Yêu cầu: Xây dựng ứng dụng chạy trên nền tảng Android từ một mã nguồn chung; tận dụng hiệu quả phần cứng thiết bị và rút ngắn thời gian phát triển đồ án.

Lựa chọn: React Native + Expo, TypeScript, định tuyến expo-router.

React Native cho phép viết giao diện bằng React và biên dịch ra các thành phần native. Việc sử dụng Expo giúp loại bỏ các bước cấu hình phức tạp trên

Xcode/Android Studio. TypeScript cung cấp kiểm soát kiểu dữ liệu tĩnh, kết hợp cùng `expo-router` giúp định tuyến màn hình trực quan theo cấu trúc thư mục.

Bảng 3.2: Các phương án nền tảng di động thay thế

Phương án	Ưu / Nhược điểm	Lý do không lựa chọn
Native thuần (Kotlin/Swift)	Hiệu năng tối đa nhưng tốn nhân lực viết hai mã nguồn độc lập	Vượt quá quy mô và thời lượng cho phép của đề án
Flutter	Hiệu năng cao nhưng sử dụng ngôn ngữ Dart	Đội ngũ phát triển đã quen thuộc với hệ sinh thái JS/React
React Native "trần" (Không dùng Expo)	Linh hoạt về cấu hình nhưng quy trình build, OTA phức tạp	Expo đã giải quyết trọn vẹn bài toán tích hợp hệ thống

3.4.2 Máy chủ phụ trợ và REST API

Yêu cầu: Cung cấp API quản lý luồng nghiệp vụ cốt lõi: bài đăng, yêu cầu nhận, luồng giao hàng và hồ sơ người dùng.

Lựa chọn: Node.js + Express 5.

Express là framework tối giản nhưng sở hữu hệ thống định tuyến và middleware vô cùng linh hoạt. Kiến trúc được phân tầng chuẩn mực (*routes* → *controllers* → *services* → *models*). Việc dùng chung ngôn ngữ lập trình giữa client và server mang lại lợi thế lớn. Thay vì sử dụng NestJS (quá cồng kềnh) hay Spring Boot/Django (khác ngôn ngữ), Node.js kết hợp cùng thư viện **Axios** ở client mang lại sự đồng bộ và hiệu năng hoàn toàn đáp ứng được quy mô đề án.

3.5 Quản lý dữ liệu và tích hợp dịch vụ

3.5.1 Cơ sở dữ liệu

Yêu cầu: Lưu trữ linh hoạt các thực thể có cấu trúc co giãn và truy vấn tốc độ cao theo trạng thái, vị trí.

Lựa chọn: MongoDB (NoSQL) + Mongoose (ODM).

Mô hình NoSQL hướng tài liệu (document-oriented) của MongoDB khớp tự nhiên với dữ liệu JSON trao đổi qua REST API [6]. Mongoose bổ sung lược đồ, giúp kiểm tra ràng buộc tham chiếu chặt chẽ ở tầng ứng dụng [7]. CSDL quan hệ (như PostgreSQL) không được ưu tiên do nghiệp vụ hệ thống ít phát sinh các giao dịch ACID phức tạp, đồng thời MongoDB mang lại sự linh hoạt

cao khi cần mở rộng thông tin hồ sơ.

3.5.2 Giao tiếp thời gian thực (Realtime)

Lựa chọn: Socket.IO.

Cung cấp kênh truyền hai chiều mạnh mẽ dựa trên WebSocket, có sẵn cơ chế tự động kết nối lại và phân phòng [8]. Giải pháp này nhẹ, tích hợp gọn gàng vào Express server, không gây độ trễ như HTTP polling định kỳ và không tốn chi phí phụ thuộc vào bên thứ ba (như Pusher).

3.5.3 Xác thực và phân quyền

Hệ thống sử dụng **JWT** lưu trữ an toàn trong **expo-secure-store** cho phiên làm việc; **bcrypt** để bảo mật mật khẩu.

3.5.4 Bản đồ và tải tệp tin

(i) **Bản đồ:** Sử dụng react-native-maps và expo-location để xin quyền thiết bị, chọn điểm và truy xuất GPS. (ii) **Tải tệp:** Kết hợp Multer xử lý dữ liệu multipart/form-data và expo-image-picker. Để tối ưu chi phí cho quy mô đồ án, hệ thống tạm thời lưu tệp cục bộ trên máy chủ thay vì dùng AWS.

3.5.5 Tác vụ định kỳ và quản lý trạng thái

(i) **Tác vụ tự động:** Sử dụng bộ hẹn giờ setInterval của Node.js để chạy ngầm các tiến trình tự động (hủy đơn quá hạn, tự xác nhận đơn sau 24h) mà không cần tích hợp bộ lập lịch công kênh như node-cron. (ii) **Trạng thái Client:** Áp dụng Zustand để quản lý trạng thái (gọn nhẹ, ít mã rập khuôn hơn Redux) cùng một mô-đun đa ngôn ngữ i18n tự xây dựng, tránh phụ thuộc vào thư viện bên ngoài.

3.6 Kết luận

Backend được đóng gói vào các ảnh container bằng **Docker** nhằm đảm bảo môi trường vận hành nhất quán tuyệt đối giữa lúc phát triển và lúc đưa lên máy chủ thực tế. Quyết định lựa chọn công nghệ bám sát ba tiêu chí: (i) Thống nhất ngôn ngữ JS/TS xuyên suốt hệ thống; (ii) Lựa chọn giải pháp "đủ dùng", tài liệu dồi dào, phù hợp với thời lượng đồ án; (iii) Giải quyết trực tiếp các bài toán ở Chương 2. Các phương án thay thế đều đã được cân nhắc kỹ lưỡng và hệ thống được thiết kế mở, hoàn toàn sẵn sàng tích hợp các dịch vụ đám mây tiên tiến khi cần mở rộng quy mô.